



MÓDULO 1 · T04

Estado y Eventos en React Native

⚡ *Interactividad con `useState`*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión aprenderás a hacer tus componentes interactivos mediante el hook `useState` y los manejadores de eventos táctiles. Al terminar, serás capaz de gestionar estado local, reaccionar a la interacción del usuario y cambiar estilos dinámicamente.

1

Gestionar estado local

Usar `useState` para almacenar y actualizar valores dentro de un componente funcional.

2

Eventos de toque

Manejar interacciones del usuario con `onPress` y `Pressable`.

3

Controles nativos

Usar `Switch` y `Slider` con su patrón `value + onChange`.

4

Estilos dinámicos

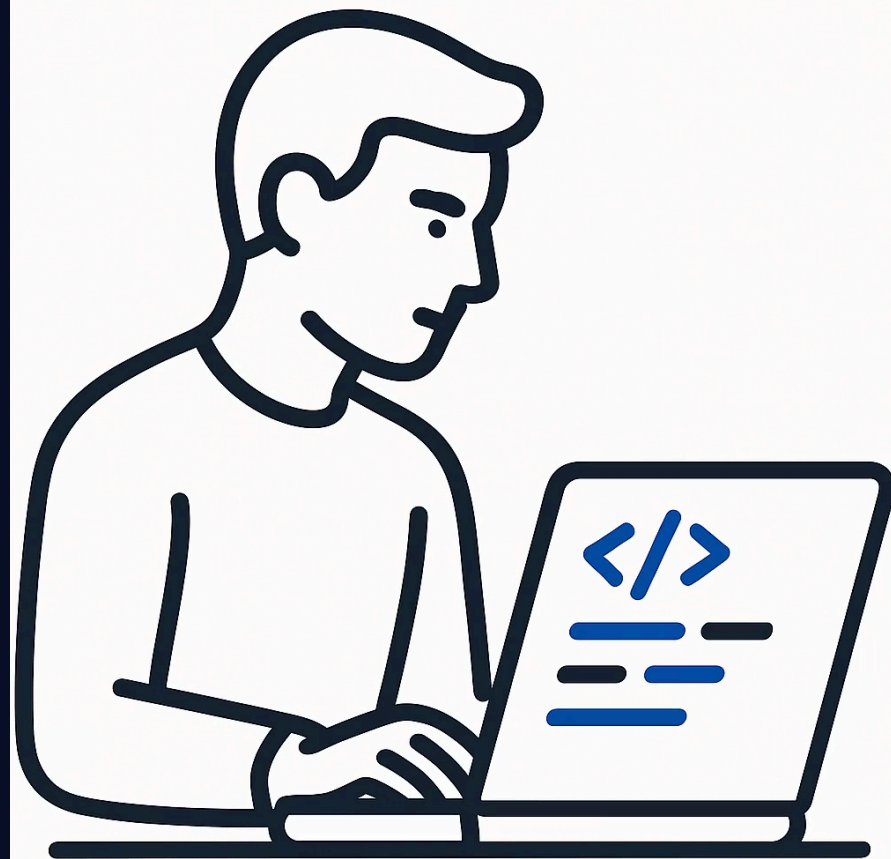
Cambiar `backgroundColor`, colores de texto y más según el valor del estado.

useState — el hook esencial

El hook `useState` devuelve un par: el valor actual del estado y una función para actualizarlo. Cada llamada a `setValor()` provoca un **re-render** del componente con el nuevo valor, manteniendo la UI siempre sincronizada con los datos.

```
const [valor, setValor] = useState<TipoDelEstado>(valorInicial);  
//   ↑      ↑           ↑  
// lectura escritura valor inicial
```

- ❗ Llama a los hooks siempre en el nivel superior del componente – nunca dentro de condicionales o bucles. Esta regla garantiza que el orden de los hooks sea estable entre renders.



Ejemplo básico — contador

¿Cómo funciona?

El estado `count` vive dentro del componente. Cada vez que el usuario pulsa + o -, `setCount` actualiza el valor y React vuelve a renderizar el componente mostrando el número actualizado.

No hace falta manipular el DOM ni buscar el elemento – React se encarga de reflejar el nuevo estado en pantalla automáticamente.

- ✓ El patrón `setCount(count + 1)` es la forma más directa de actualizar estado derivado del valor anterior.

Código del contador

```
export default function Contador() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <View>  
      <Text style={{ fontSize: 48 }}>  
        {count}  
      </Text>  
      <Button  
        title="+"  
        onPress={() => setCount(count + 1)}  
      />  
      <Button  
        title="-"  
        onPress={() => setCount(count - 1)}  
      />  
    </View>  
  );  
}
```

Switch y Slider

Los componentes `Switch` y `Slider` siguen siempre el mismo patrón en React Native: reciben el valor actual a través de `value` y notifican los cambios a través de `onValueChange`. El estado vive en el componente padre y ellos solo lo reflejan.

```
const [modoOscuro, setModoOscuro] = useState(false);
```

```
<Switch  
  value={modoOscuro}  
  onValueChange={setModoOscuro}  
  trackColor={{ true: '#EF5350', false: '#ccc' }}  
  thumbColor={modoOscuro ? '#fff' : '#fff'}  
/>
```

❗ `onValueChange` recibe directamente el nuevo booleano, por lo que puedes pasarle `setModoOscuro` sin necesidad de una función flecha intermediaria.

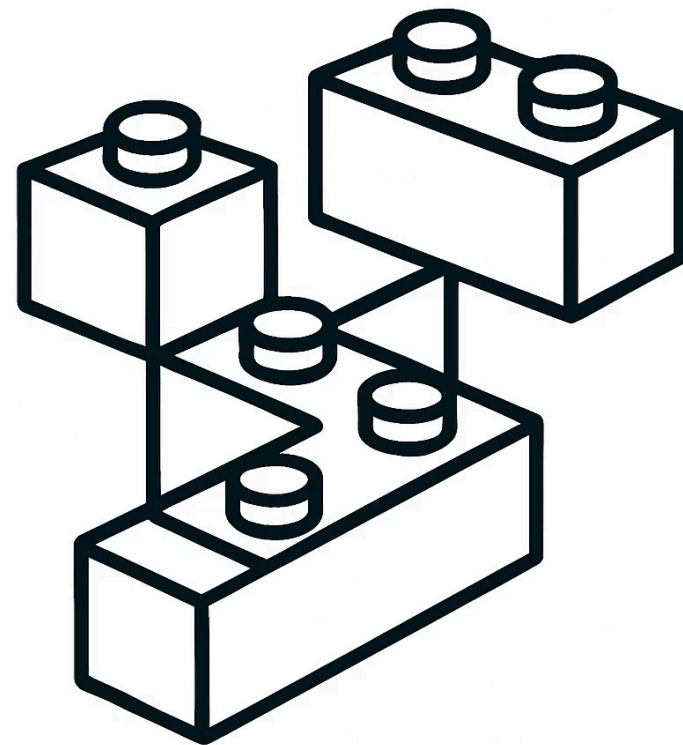
Estilo dinámico

Una de las aplicaciones más habituales de `useState` es cambiar estilos en función del estado. En React Native los estilos son objetos JavaScript, por lo que puedes calcularlos con expresiones condicionales antes del `return`.

```
const [oscuro, setOscuro] = useState(false);

const bg = oscuro ? '#121212' : '#F5F5F5';
const text = oscuro ? '#EFEFEF' : '#212121';

<View style={[styles.screen, { backgroundColor: bg }]}>
  <Text style={{ color: text }}>Hola</Text>
  <Switch
    value={oscuro}
    onChange={setOscuro}
  />
</View>
```



Múltiples estados

```
const [modoOscuro, setModoOscuro] = useState(false);  
const [volumenMusica, setVolumenMusica] = useState(70);  
const [volumenFx, setVolumenFx] = useState(80);
```

⚠ Mantén los estados lo más cerca posible del componente que los usa. Si varios componentes los necesitan → súbelos al padre o usa Context.

PokeApp — SettingsScreen.tsx

Este es el componente simplificado tal y como aparece en la aplicación de ejemplo. Observa cómo el `backgroundColor` del `ScrollView` se calcula directamente a partir del estado `modoOscuro`, y cómo el volumen se delega al subcomponente `VolumeRow` mediante props.

```
export default function SettingsSliders() {
  const [modoOscuro, setModoOscuro] = useState(false);
  const [volumen, setVolumen] = useState(70);

  return (
    <ScrollView
      style={{ backgroundColor: modoOscuro ? '#121' : '#F5F5F5' }}
    >
      <Switch
        value={modoOscuro}
        onChange={setModoOscuro}
      />
      <Text>Volumen: {volumen}%</Text>
      <VolumeRow valor={volumen} onChange={setVolumen} />
    </ScrollView>
  );
}
```

📌 **Buena práctica:** calcula los valores derivados del estado (colores, textos) justo antes del `return`, no dentro del JSX, para mantener el marcado limpio y legible.

Tour starter/SettingsSliders.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica con los dos controles marcados como `TODO` que deberás completar durante el ejercicio. El estado aún no existe y los controles no están conectados.

```
export default function SettingsSliders() {  
  // TODO: useState aquí  
  
  return (  
    <ScrollView style={styles.screen}>  
      <View style={styles.row}>  
        <Text style={styles.label}>Modo oscuro</Text>  
        { /* TODO: Switch */ }  
      </View>  
      <View style={styles.row}>  
        <Text style={styles.label}>Volumen</Text>  
        { /* TODO: controles de volumen */ }  
      </View>  
    </ScrollView>  
  );  
}
```

⚠ No modifiques los estilos base del `StyleSheet` – céntrate en los `TODO` marcados arriba para añadir estado y conectar los controles.

✓ SOLUCIÓN

Solución — solucion/SettingsSliders.tsx

La solución completa añade los estados necesarios y conecta cada control a su setter correspondiente. El `backgroundColor` del contenedor cambia dinámicamente y los botones de volumen usan `Math.min` / `Math.max` para mantener el valor entre 0 y 100.

```
const [modoOscuro, setModoOscuro] = useState(false);
const [volumenMusica, setVolumenMusica] = useState(70);

const bg = modoOscuro ? '#121212' : '#F5F5F5';

// ...
<Switch value={modoOscuro} onValueChange={setModoOscuro} />

<Pressable
  onPress={() => setVolumenMusica(v => Math.min(100, v + 10))}
>
  <Text>+</Text>
</Pressable>

<Pressable
  onPress={() => setVolumenMusica(v => Math.max(0, v - 10))}
>
  <Text>-</Text>
</Pressable>
```

✓ Si el fondo cambia al activar el Switch y los botones mueven el volumen entre 0 y 100, ¡lo has conseguido! 🎉

Ejercicio T04

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el patrón de múltiples estados y union types.

1 Abre el fichero de inicio

Navega a `starter/SettingsSliders.tsx` en tu editor.

2 Añade estado `modoOscuro`

Declara `const [modoOscuro, setModoOscuro] = useState(false)` y conéctalo al `Switch`.

3 Añade estado `volumen`

Declara `const [volumen, setVolumen] = useState(70)` y añade botones `+` / `-` que cambien el valor de 10 en 10.

4 Cambia el fondo dinámicamente

Usa `modoOscuro` para calcular el `backgroundColor` del contenedor principal.

5 Bonus: estado `modo`

Añade estado `modo: 'system' | 'light' | 'dark'` con 3 botones `Pressable` que lo cambien.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá crear componentes React Native interactivos, con estado local bien gestionado y estilos que responden al comportamiento del usuario.

Concepto	Sintaxis clave	Para qué sirve
useState	<code>const [v, setV] = useState(init)</code>	Estado local en componentes funcionales
setValor	<code>setValor(nuevoValor)</code>	Dispara re-render con el nuevo valor
onPress	<code><Button onPress={() => ...} /></code>	Manejador de toque
Switch	<code>value + onChange</code>	Control booleano nativo
Estilo dinámico	<code>backgroundColor: oscuro ? '#111' : '#FFF'</code>	UI reactiva al estado
Múltiples states	Un <code>useState</code> por valor independiente	Separación clara de responsabilidades

✅ ¡Enhorabuena! Ahora tienes las herramientas para crear componentes interactivos con estado local, eventos táctiles y estilos dinámicos en React Native. 🚀